

Instituto Superior de Engenharia de Coimbra

Programação WEB

2025/2026

Trabalho realizado por:

David Oliveira - 2022147017

Pedro Freitas - 2022149070

Índice

1. Introdução 3

1.1. Enquadramento e Objetivos 3

1.2. Credenciais de Acesso 3

2. Análise de Requisitos 4

2.1. Atores do Sistema 4

2.2. Regras de Negócio 4

2.3. Requisitos Funcionais 6

3. Arquitetura e Tecnologias 7

3.1. Arquitetura da Solução 7

3.2. Tecnologias Utilizadas 9

4. Modelação de Dados 9

4.1. Modelo Entidade-Relacionamento (MER) 9

4.2. Dicionário de Dados 10

5. Implementação e Soluções Técnicas 11

5.1. Gestão Transacional de Stocks e Expedição 11

5.2. Algoritmo de Pesquisa Hierárquica de Categorias 11

5.3. Segurança e Autorização (JWT) 12

6. Interface do Utilizador 12

6.1. Loja e Catálogo (Cliente e Visitante) 12

6.2. Área do Fornecedor (Backoffice) 14

6.3. Área de Administração (Gestão) 15

7. Conclusão 17

7.1. Grau de Completude 17

7.2. Principais Dificuldades e Superação 17

7.3. Análise Autocrítica 18

1. Introdução

1.1. Enquadramento e Objetivos

O presente relatório descreve o desenvolvimento da plataforma **"MyMedia"**, realizado no âmbito da unidade curricular de Programação Web. O objetivo primordial deste trabalho foi conceber e implementar uma solução completa de comércio eletrónico (Marketplace) que simule um cenário real de interação entre **Administradores**, **Fornecedores** e **Clientes**.

Para concretizar este propósito, adotou-se uma arquitetura distribuída moderna baseada no ecossistema **.NET 8**, separando claramente a lógica de negócio e acesso a dados (**Backend API REST**) da interface de utilizador (**Frontend em Blazor**). Esta abordagem permitiu não só cumprir os requisitos funcionais, mas também garantir a segurança e escalabilidade da aplicação.

Os objetivos específicos centraram-se na resolução de desafios complexos de gestão e engenharia de software, nomeadamente:

- A implementação de um **ciclo de vida do produto**, desde a submissão pelo fornecedor (estado Pendente) até à aprovação e definição de margem de lucro pelo administrador (estado Ativo).
- A criação de um **catálogo dinâmico** com categorização hierárquica, permitindo pesquisas que abrangem categorias pai e filhas.
- O desenvolvimento de um sistema de **compras e gestão de stock transacional**, garantindo que as quantidades e estados das encomendas são atualizados de forma consistente e segura na base de dados apenas após a validação administrativa.

1.2. Credenciais de Acesso

Para efeitos de avaliação e teste das funcionalidades implementadas nos diferentes perfis de utilizador, devem ser utilizadas as seguintes credenciais:

Perfil	Email (Login)	Password	Funções Principais
Administrador	admin@mymedia.com		Gestão global, aprovação de produtos, expedição de encomendas e gestão de utilizadores.
Fornecedor	ana@santos.com	Password123!	Submissão de produtos e visualização das vendas dos seus artigos.
Cliente	joao@teste.com	Password123!	Acesso à loja, gestão do carrinho de compras e histórico de encomendas.

2. Análise de Requisitos

A análise de requisitos é uma fase fundamental no desenvolvimento de software, definindo o comportamento esperado do sistema e as restrições sob as quais este deve operar. Para o projeto "MyMedia", foram identificados os seguintes atores, regras de negócio e requisitos funcionais.

2.1. Atores do Sistema

O sistema foi desenhado para suportar quatro tipos de intervenientes, cada um com níveis de acesso e responsabilidades distintos:

- **Visitante:** Utilizador não autenticado. Tem acesso apenas à visualização do catálogo público de produtos, sem acesso a preços finais ou à funcionalidade de compra.
- **Cliente:** Utilizador registado que usufrui dos serviços da loja. Pode adicionar produtos ao carrinho, finalizar encomendas e consultar o seu histórico de compras.
- **Fornecedor:** Utilizador parceiro que submete produtos para venda na plataforma. Tem acesso a uma área reservada para gestão do seu catálogo e consulta das vendas associadas aos seus artigos.
- **Administrador (Gestão):** Utilizador com privilégios elevados (Administrador/Funcionário). É responsável pela manutenção do sistema, aprovação de novos utilizadores e produtos, definição de margens de lucro e gestão logística das encomendas.

2.2. Regras de Negócio

Para garantir a integridade dos dados e o cumprimento do modelo de negócio proposto, foram implementadas as seguintes regras estritas no Backend (API):

RN01 - Ciclo de Vida e Aprovação de Produtos Todo o produto criado por um Fornecedor entra, por defeito, no estado **"Pendente"**. Este produto permanece invisível no catálogo público até que um Administrador proceda à sua validação. Durante o processo de aprovação, o Administrador é obrigado a definir uma **Margem de Lucro (%)**, que incide sobre o Preço Base para calcular o Preço Final de venda ao público. Só após este passo o produto transita para o estado **"Ativo"**.

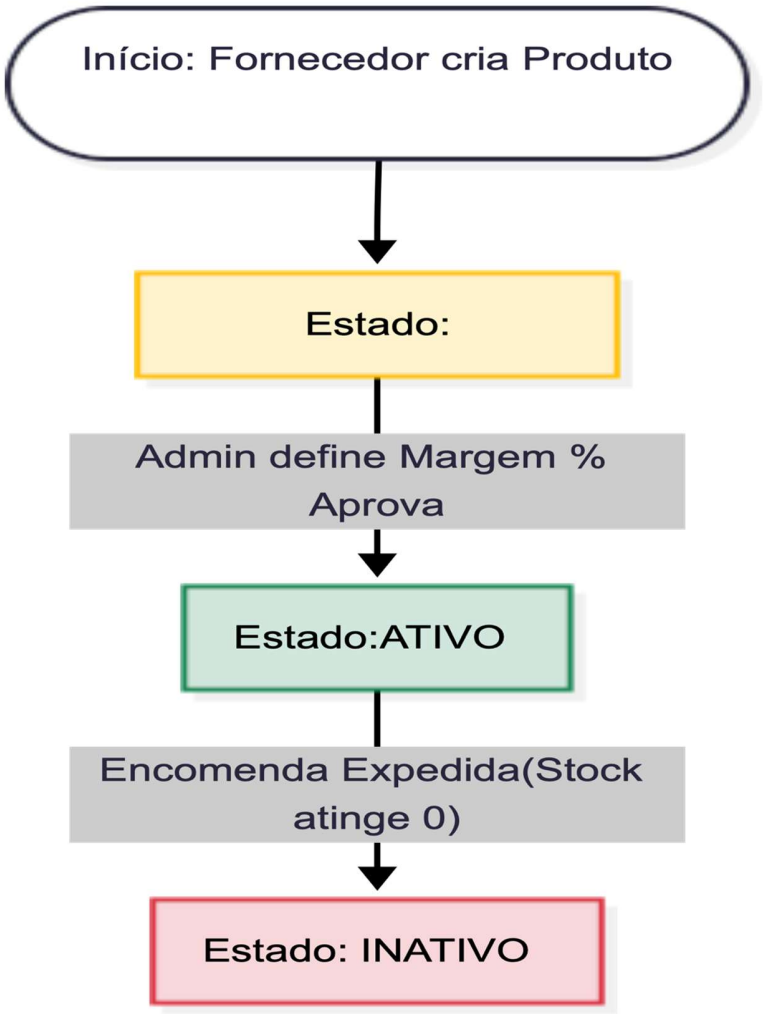


Figura 1: Diagrama de estados do ciclo de vida de um produto na plataforma MyMedia.

RN02 - Gestão Transacional de Stock A gestão de inventário segue uma política de "reserva tardia".

1. No momento do *checkout* (compra), o sistema valida se existe quantidade suficiente, mas **não decrementa** o stock na base de dados, garantindo que encomendas não pagas ou pendentes não bloqueiam inventário.
2. O decremento efetivo do stock ocorre apenas quando o Administrador altera o estado da encomenda para **"Expedida"**.
3. Caso a quantidade de um produto atinja zero (0), após a expedição, o sistema altera automaticamente o estado do produto para **"Inativo"**, removendo-o do catálogo.

RN03 - Isolamento de Dados. O sistema garante a privacidade através de filtros ao nível da base de dados:

- Um **Fornecedor** apenas consegue visualizar e editar os produtos que ele próprio criou.
- Um **Cliente** apenas tem acesso aos detalhes das encomendas que ele próprio realizou.

RN04 - Hierarquia de Categorias A categorização dos produtos suporta múltiplos níveis (Categorias Pai, Filhas e Netas). A regra de visualização impõe que, ao filtrar por uma Categoria Pai (ex: "Filmes"), o sistema deve apresentar recursivamente todos os produtos dessa categoria e de todas as suas descendentes.

2.3. Requisitos Funcionais

Os requisitos funcionais descrevem as operações específicas que o sistema executa para satisfazer as necessidades dos atores.

Módulo de Autenticação e Utilizadores:

- **RF01:** Registo de novos utilizadores (Clientes e Fornecedores) com atribuição automática de perfil.
- **RF02:** Autenticação via JWT (JSON Web Tokens) para segurança da API.
- **RF03:** Aprovação manual de contas de Fornecedores por parte do Administrador.

Módulo de Catálogo e Loja (Frontend):

- **RF04:** Visualização de produtos em destaque e listagem paginada.
- **RF05:** Pesquisa de produtos por nome e filtragem por árvore de categorias.
- **RF06:** Gestão de Carrinho de Compras (adicionar, remover, alterar quantidades) com persistência local.
- **RF07:** Criação de Encomendas (*Checkout*).

Módulo de Gestão (Backoffice - Admin):

- **RF08:** Listagem e validação de produtos pendentes (definição de preços).
- **RF09:** Gestão de Encomendas (Visualização de detalhe e alteração de estado para "Expedida").
- **RF10:** Gestão de Utilizadores (Listar, Ativar/Desativar contas).
- **RF11:** CRUD de Categorias e Modos de Entrega.

Módulo de Fornecedor (Backoffice - Fornecedor):

- **RF12:** Criação e edição de produtos próprios.
- **RF13:** Consulta de listagem de vendas (produtos seus que foram encomendados).

3. Arquitetura e Tecnologias

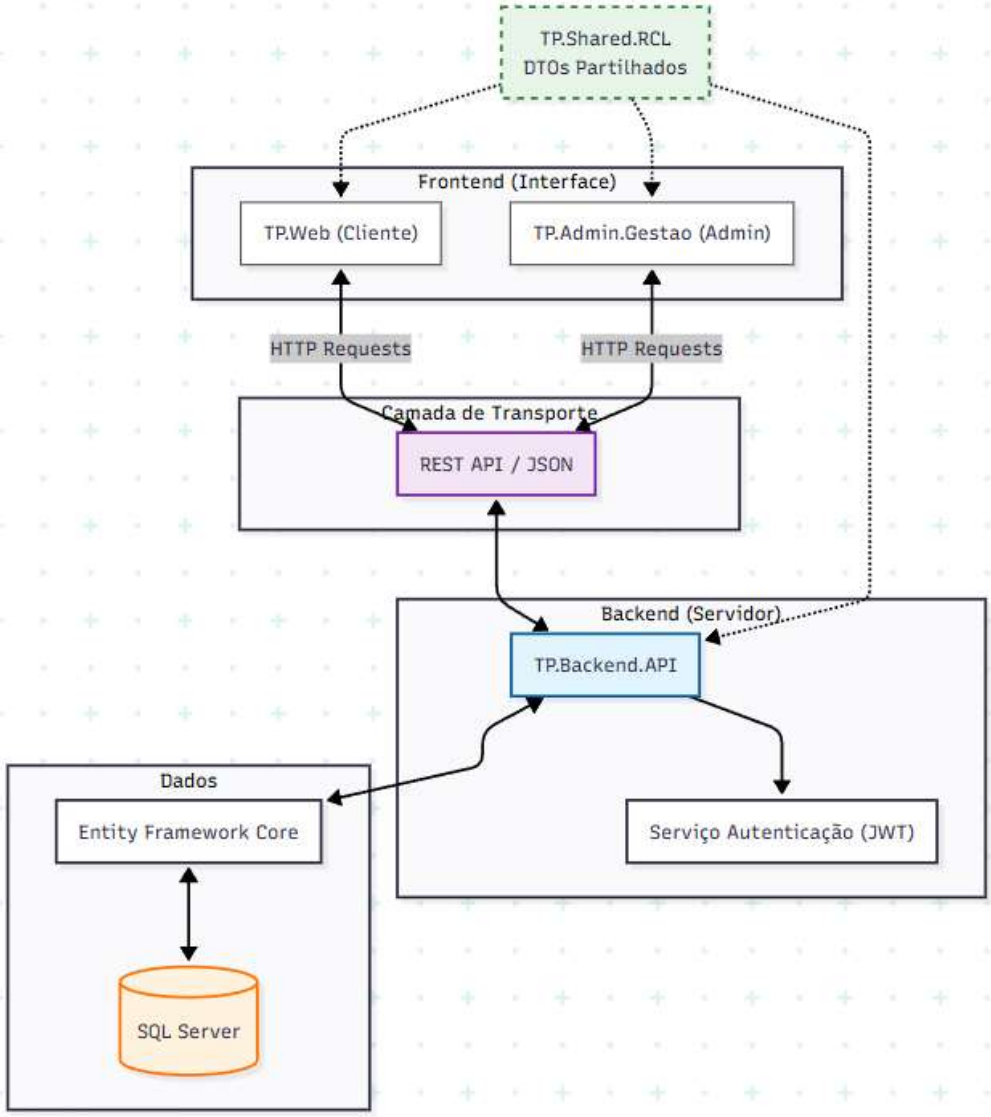
A arquitetura de software define a organização fundamental do sistema, os seus componentes e as relações entre eles. Para o projeto "MyMedia", optou-se por uma arquitetura distribuída e modular, baseada no padrão **Client-Server**, separando totalmente a lógica de negócio (Backend) da interface de apresentação (Frontend).

3.1. Arquitetura da Solução

A solução foi estruturada em múltiplos projetos dentro da mesma solução Visual Studio, garantindo a separação de responsabilidades. A comunicação entre os Frontends e o Backend é realizada exclusivamente através de pedidos HTTP (REST/JSON), garantindo o desacoplamento entre camadas.

A estrutura física da solução é composta pelos seguintes componentes principais:

1. **TP.Backend.API (Servidor):**
   - Atua como o núcleo do sistema. Expõe uma API RESTful segura.
   - Não possui interface gráfica (apenas Swagger para documentação).
   - Responsável pela validação de dados, regras de negócio (ex: cálculo de stocks), autenticação e autorização via JWT.
   - Comunica com a base de dados através do Entity Framework Core.
2. **TP.Backend.Data (Persistência):**
   - Biblioteca de classes (Class Library) que contém o contexto da base de dados (ApplicationDbContext) e as entidades do domínio (Tabelas).
   - Isola o acesso a dados, permitindo que possa ser reutilizado por outros serviços se necessário.
3. **TP.Shared.RCL (Partilhado):**
   - Biblioteca de Classes Razor (RCL) que contém os **DTOs** (Data Transfer Objects) partilhados.
   - **Vantagem:** Permite que o Backend e o Frontend usem as mesmas classes (ex: ProdutoDto, EncomendaDto) para comunicar, evitando duplicação de código e erros de tipagem.
4. **TP.Web (Frontend Cliente):**
   - Aplicação Blazor WebAssembly (ou Server) focada na experiência do consumidor final.
   - Consome a API para listar produtos e registar encomendas.
5. **TP.Admin.Gestao (Frontend Administração):**
   - Aplicação Blazor dedicada à gestão de backoffice.
   - Permite operações privilegiadas como aprovação de produtos, gestão de utilizadores e expedição de encomendas.



A **Figura 2:** ilustra o fluxo de comunicação da arquitetura implementada, onde os pedidos HTTP (JSON) servem como ponte entre as camadas.

3.2. Tecnologias Utilizadas

A seleção tecnológica privilegiou ferramentas modernas, robustas e com suporte a longo prazo (LTS), alinhadas com o ecossistema Microsoft.

- **Plataforma:** .NET 8 (C#). Escolhida pela sua performance, tipagem forte e vasta biblioteca de classes base.
- **API Framework:** ASP.NET Core Web API. Utilizada para criar os *endpoints* REST, gerir injeção de dependências e *middleware* de segurança.
- **Interface Web: Blazor**.
    - Permitiu o desenvolvimento de *Single Page Applications* (SPA) interativas utilizando C# em vez de JavaScript, uniformizando a linguagem em todo o projeto.
- **ORM (Object-Relational Mapping): Entity Framework Core**.
    - Utilizou-se a abordagem **Code-First**, onde as tabelas da base de dados são geradas automaticamente a partir das classes C# (Entidades), facilitando a evolução do esquema através de Migrações (Migrations).
- **Base de Dados:** Microsoft SQL Server (LocalDB). Sistema de gestão de base de dados relacional robusto e integrado nativamente com o Visual Studio.
- **Autenticação: JWT (JSON Web Tokens)**.
    - Implementação de autenticação *stateless*. O servidor emite um token assinado criptograficamente no login, que o cliente envia em cada pedido subsequente no cabeçalho Authorization, garantindo segurança sem necessidade de sessões no servidor.
- **Design:** Bootstrap 5. Utilizado para garantir a responsividade da aplicação (adaptação a telemóveis e desktops) e uma estética limpa e funcional.

4. Modelação de Dados

A persistência de dados do projeto "MyMedia" é suportada pelo Microsoft SQL Server. A estrutura da base de dados foi definida utilizando a abordagem **Code-First** do Entity Framework Core, onde o esquema relacional é gerado automaticamente a partir das classes de domínio (Entidades) definidas na aplicação.

4.1. Modelo Entidade-Relacionamento (MER)

O diagrama seguinte ilustra as principais entidades do sistema e a relação entre elas. Destaca-se a centralização do ApplicationUser (que representa tanto Clientes como Fornecedores) e a estrutura hierárquica das Categorias.

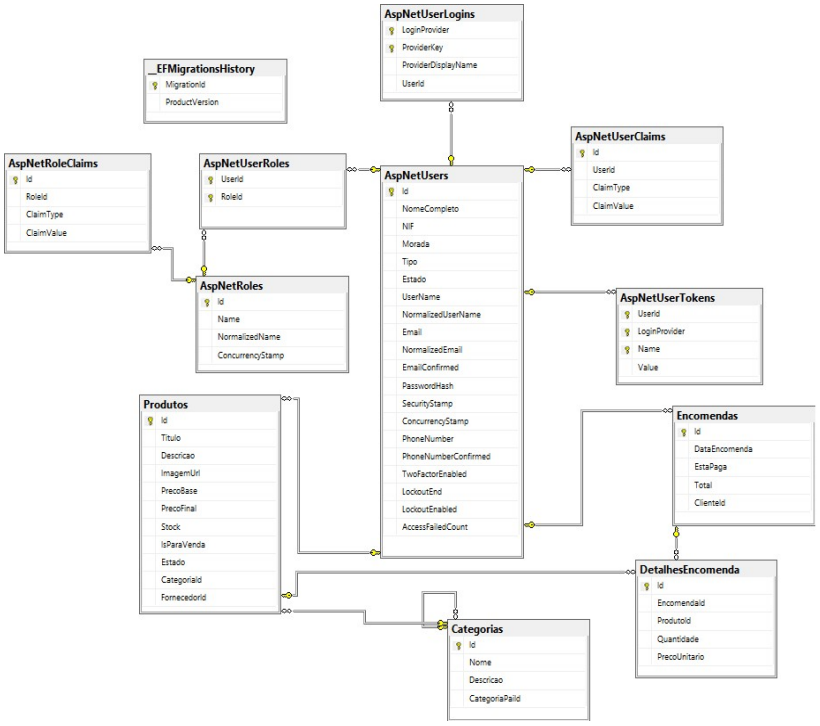


Figura 3: Modelo Entidade-Relacionamento (MER) gerado a partir do SQL Server.

4.2. Dicionário de Dados

Abaixo descrevem-se as tabelas fundamentais que suportam a lógica de negócio do sistema.

Tabela: AspNetUsers (ApplicationUser) Estende a tabela padrão do sistema de Identidade da Microsoft. Armazena todos os intervenientes (Administradores, Fornecedores e Clientes).

- **Campos Extra:** Foram adicionados campos como NIF e Morada para efeitos de faturação e envio, bem como um campo discriminador para distinguir o perfil do utilizador.

Tabela: Produtos Armazena o catálogo da loja. Possui relações importantes:

- **Relação com Fornecedor:** Chave estrangeira para AspNetUsers, permitindo saber quem vende o quê.
- **Gestão de Preço:** Armazena o PrecoBase (do fornecedor) e a Margem (do admin) separadamente, permitindo recálculos futuros.

- **Estado:** Campo enumerado (Pendente, Ativo, Inativo) que controla a visibilidade do produto na loja.

Tabela: Categorias Tabela recursiva que permite criar uma árvore de categorias infinita.

- **Auto-relacionamento:** O campo CategoriaPaiId é uma chave estrangeira que aponta para a própria tabela Categorias. Se for NULL, é uma categoria de topo (ex: "Filmes"). Se tiver valor, é uma subcategoria (ex: "Ação").

Tabela: Encomendas e LinhasEncomenda Representam a transação comercial.

- **Cabeçalho (Encomenda):** Guarda o total, a data e o estado global (Pendente, Expedida).
- **Detalhe (Linhas):** Guarda o "snapshot" do preço no momento da compra. Isto é crucial: se o Fornecedor alterar o preço do produto *após* a venda, o registo histórico na LinhaEncomenda mantém o preço original da transação.

5. Implementação e Soluções Técnicas

Este capítulo detalha a concretização dos requisitos funcionais, focando-se nos algoritmos e na lógica de negócio implementados na API (Backend) para garantir a integridade e consistência da aplicação.

5.1. Gestão Transacional de Stocks e Expedição

Um dos maiores desafios num sistema de e-commerce é a gestão de inventário. Optou-se por não descontar o stock no momento da criação da encomenda (Checkout), para evitar que encomendas pendentes ou abandonadas "bloqueassem" stock indefinidamente.

A lógica de decremento foi centralizada no **Backend (EncomendasController)**, especificamente no método de alteração de estado para "Expedida". Este processo segue uma transação atómica, ou seja, o conjunto de operações é tratado como uma unidade indivisível, que executa três passos críticos:

1. **Verificação:** Confirma se ainda existe stock suficiente para todos os itens.
2. **Decremento:** Subtrai as quantidades.
3. **Inativação:** Se o stock de um produto atingir zero, o seu estado passa automaticamente para Inativo.

5.2. Algoritmo de Pesquisa Hierárquica de Categorias

Para permitir uma navegação intuitiva, o sistema implementa uma estrutura de categorias em árvore (Pai > Filho > Neto). O desafio técnico residia em filtrar produtos. Se um utilizador selecionar

uma categoria, espera ver também os produtos dessa subcategoria, e não apenas os que estão diretamente ligados à raiz.

Para resolver isto, implementou-se um método recursivo no Backend que, antes de efetuar a query à base de dados, recolhe todos os IDs da árvore genealógica da categoria selecionada.

Lógica implementada:

1. Recebe o ID da categoria selecionada
2. Procura recursivamente todos os IDs descendentes (filhos e filhos dos filhos).
3. Executa a pesquisa SQL filtrando por essa lista de IDs: `Query.Where(p => idsDescendentes.Contains(p.CategoriaId))`

Esta abordagem garante que a navegação é fluída e que nenhum produto fica "escondido" em subníveis.

5.3. Segurança e Autorização (JWT)

A segurança da API foi implementada utilizando **JSON Web Tokens (JWT)**.

- **Emissão:** No momento do login, o servidor valida as credenciais e emite um token assinado contendo os *Claims* do utilizador (ID, Email e Role: Cliente, Fornecedor ou Admin).
- **Transporte:** O Frontend (Blazor) armazena este token no LocalStorage e injeta-o automaticamente no cabeçalho `Authorization: Bearer <token>` de cada pedido HTTP utilizando um DelegatingHandler.
- **Proteção:** Os *endpoints* da API estão decorados com o atributo `[Authorize(Roles = "...")]`, garantindo que, por exemplo, apenas um utilizador com a Claim "Admin" consegue aceder ao *endpoint* de aprovação de produtos.

6. Interface do Utilizador

A interface da aplicação foi desenvolvida utilizando **Blazor (Razor Components)** e **Bootstrap 5**, garantindo uma experiência de utilização reativa, intuitiva e adaptável a diferentes dimensões de ecrã (*responsive design*). A navegação foi estruturada de forma a separar claramente o ambiente de loja pública (Front-office) das áreas de gestão (Back-office).

6.1. Loja e Catálogo (Cliente e Visitante)

A página principal serve como ponto de entrada para todos os utilizadores. Apresenta um *layout* de duas colunas:

1. **Lateral Esquerda:** Árvore de categorias navegável e filtros.
2. **Área Central:** Grelha de produtos com paginação.

Destaca-se a componente de "Produto em Destaque" no topo, que sugere aleatoriamente um artigo aos visitantes.

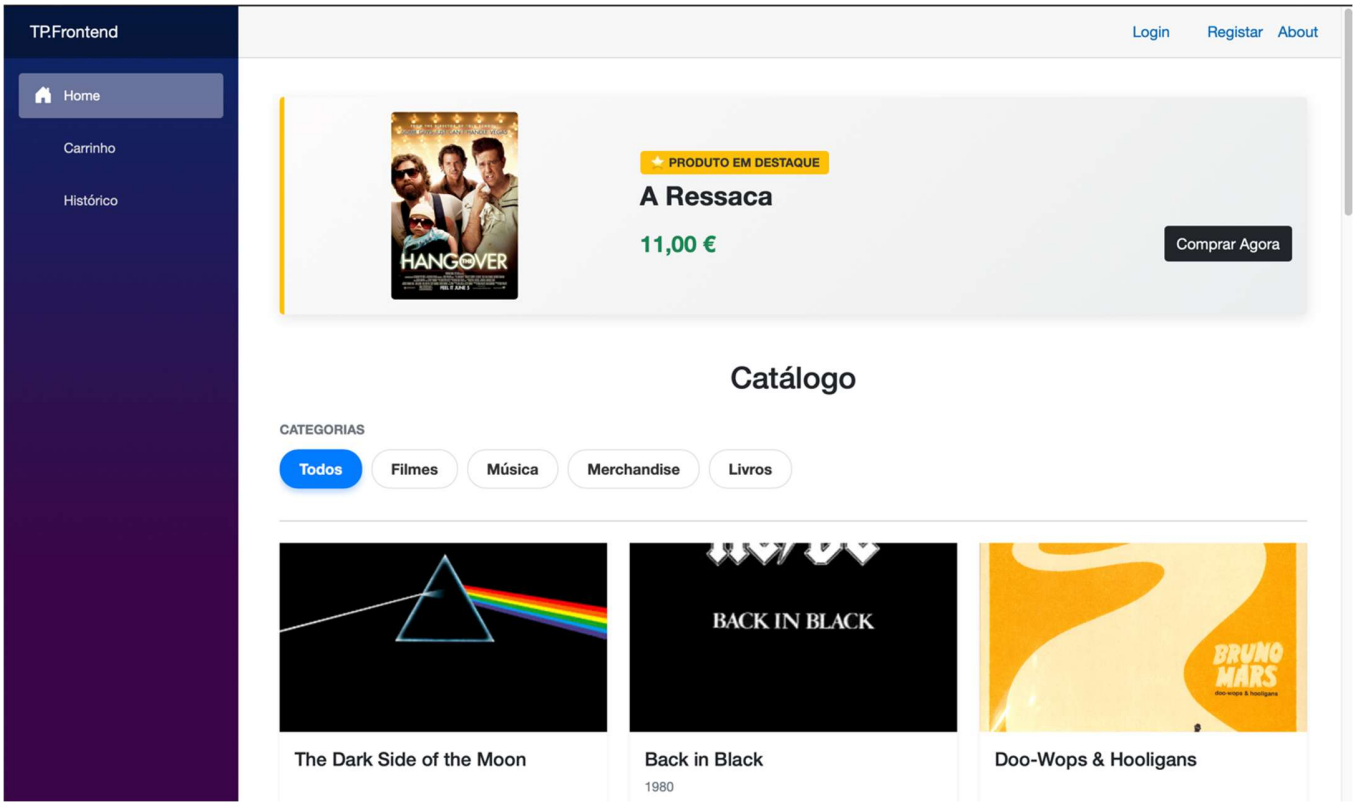


Figura 4: Página inicial da loja com catálogo de produtos e barra lateral de categorias

Ao selecionar um produto, o utilizador é direcionado para a página de detalhes, onde pode visualizar a descrição completa, o stock disponível e adicionar o artigo ao Carrinho de Compras. O estado do carrinho é permanente durante a sessão.

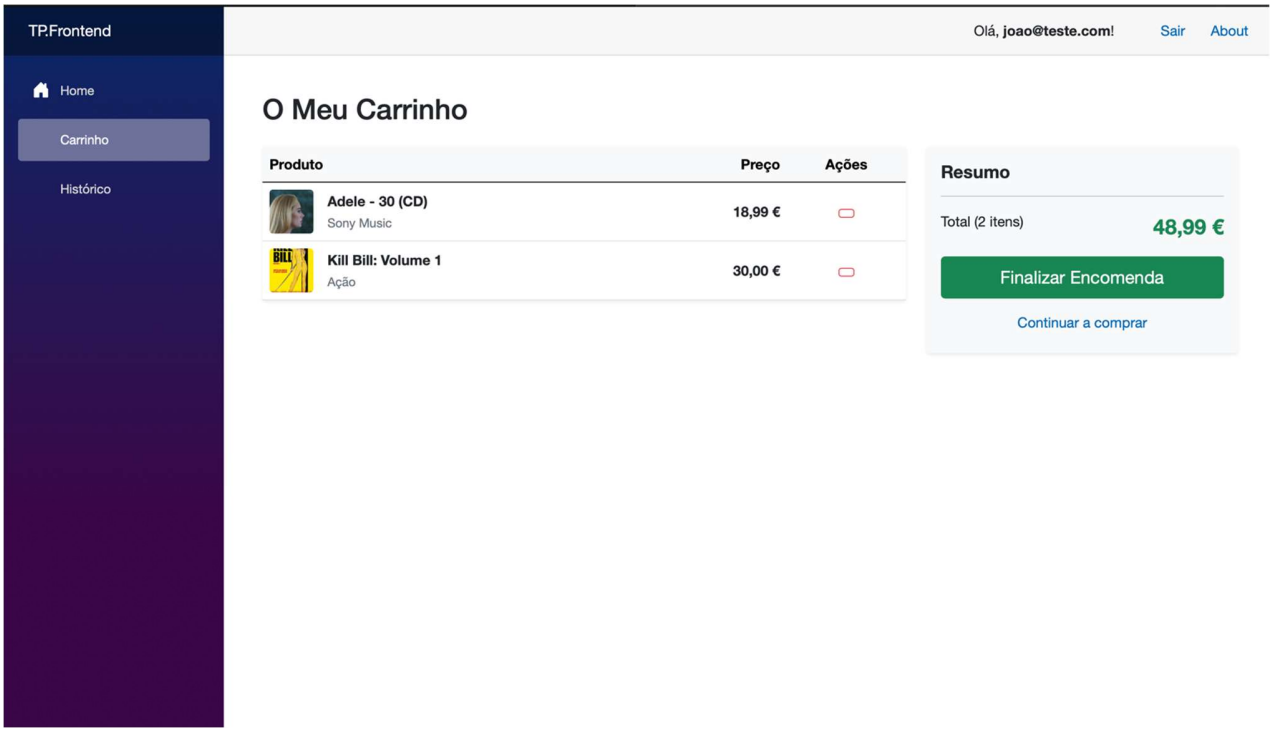


Figura 5: Detalhe de produto e gestão do carrinho de compras.

6.2. Área do Fornecedor (Backoffice)

Após autenticação, os fornecedores têm acesso a um painel dedicado onde podem gerir o seu inventário. O formulário de criação de produtos foi desenhado para ser simples, permitindo inserir título, preço base, descrição e imagem. É de notar que, após submissão, o produto fica visível na lista com o estado **"Pendente"**, aguardando aprovação.

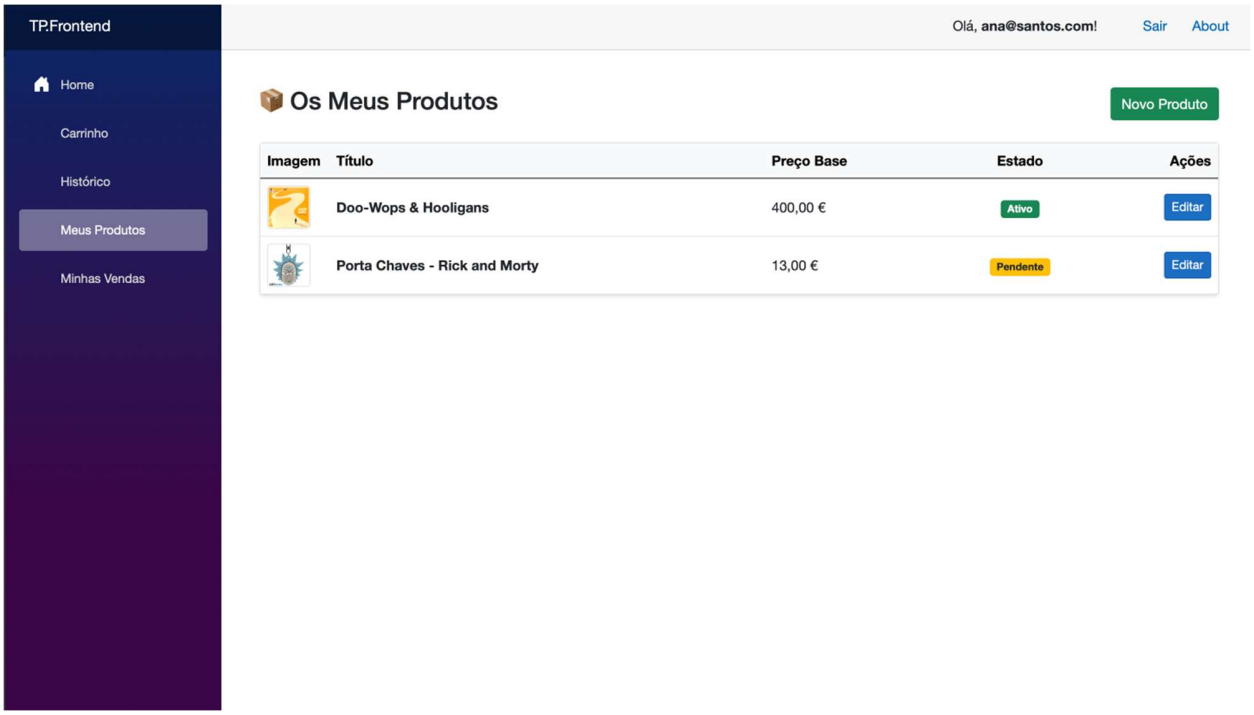


Figura 6: Backoffice do Fornecedor com listagem dos seus produtos e respetivos estados.

6.3. Área de Administração (Gestão)

O painel de administração é o centro de controlo da plataforma. Este permite aos administradores o acesso a funcionalidades críticas que não são visíveis a utilizadores comuns.

Gestão de Encomendas: Esta interface permite visualizar todas as encomendas recebidas. Conforme detalhado no capítulo anterior, é neste ecrã que o administrador valida o stock e clica no botão "Expedir", desencadeando a atualização contabilística do inventário.

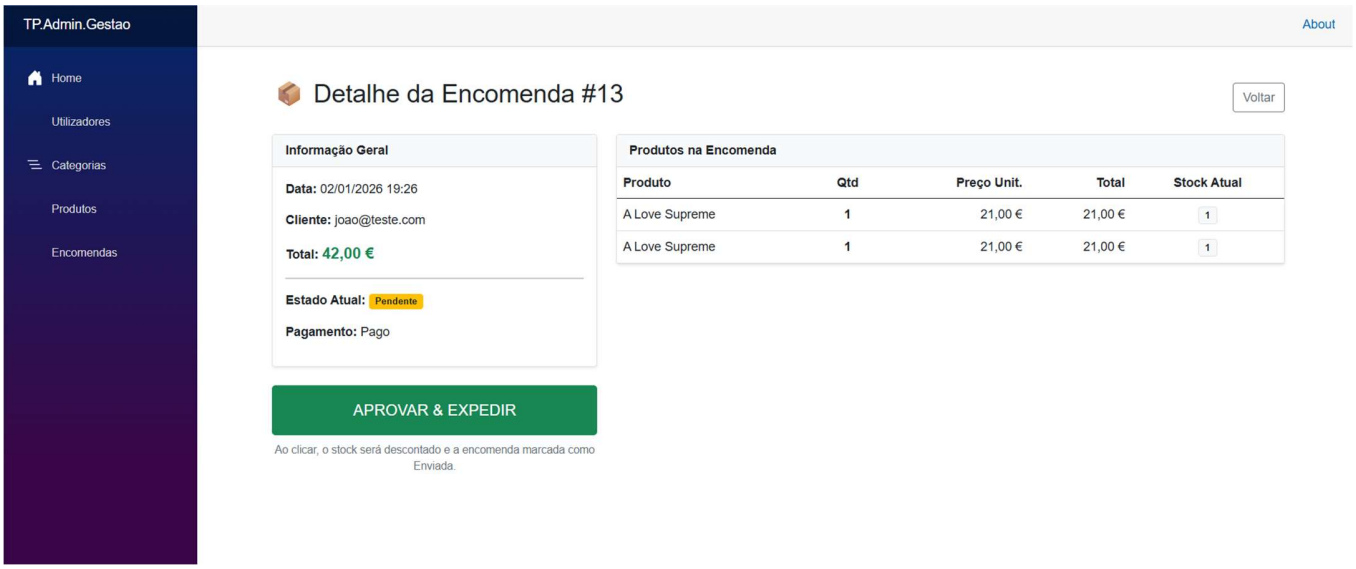


Figura 7: Interface de gestão de encomendas, permitindo a expedição e validação de stock.

Aprovação de Produtos: O administrador visualiza os produtos submetidos pelos fornecedores. Ao editar um produto pendente, o sistema obriga à definição da **Margem de Lucro (%)**, calculando automaticamente o Preço Final antes de permitir a mudança de estado para "Ativo".

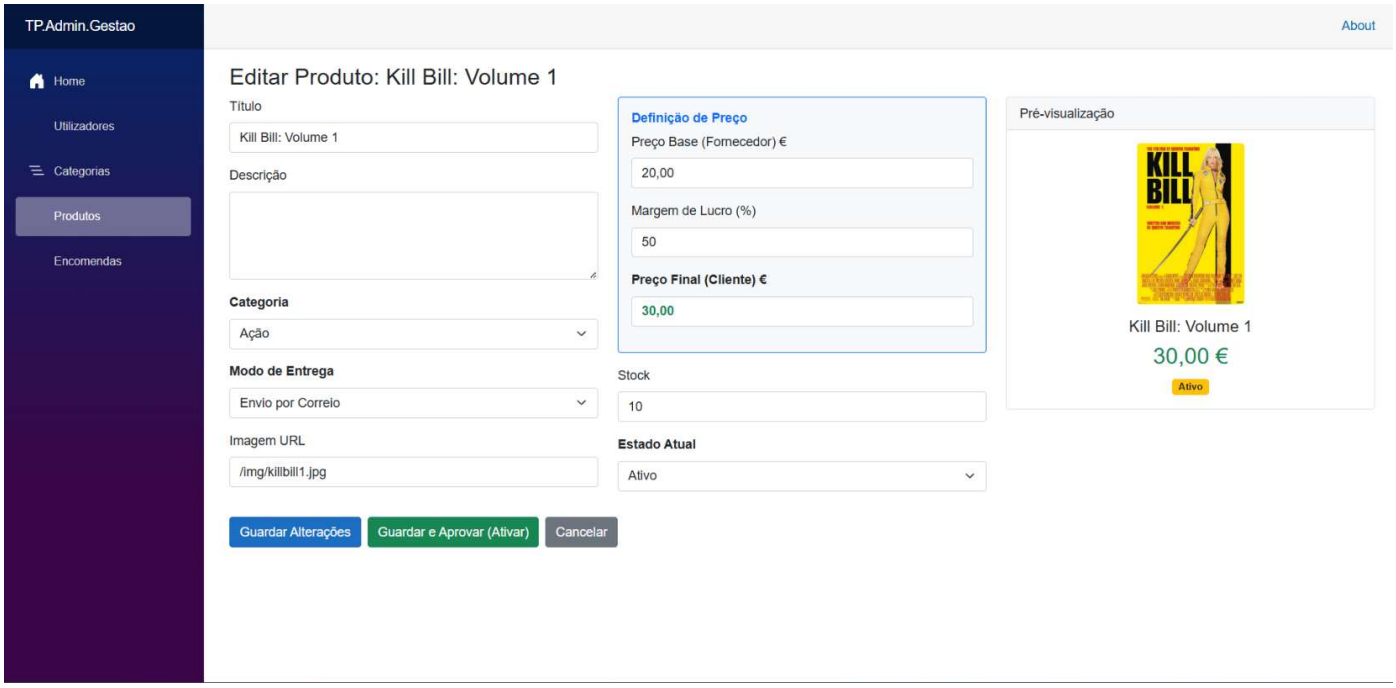


Figura 8: Fluxo de aprovação de produto com definição de margem de lucro.

7. Conclusão

O desenvolvimento da plataforma "MyMedia" constituiu um desafio técnico abrangente que permitiu aplicar e consolidar os conhecimentos adquiridos na unidade curricular de Programação Web. O resultado final é uma aplicação de comércio eletrónico funcional, robusta e baseada nas melhores práticas da arquitetura de software moderna com .NET 8.

7.1. Grau de Completude

Pode afirmar-se que os objetivos fundamentais propostos no enunciado foram **integralmente alcançados**. A aplicação suporta com sucesso o ciclo completo de uma venda online, desde o registo dos intervenientes (Clientes e Fornecedores) até à expedição da encomenda.

Destacam-se como **funcionalidades totalmente implementadas**:

- Sistema de autenticação e autorização com perfis distintos (Admin, Fornecedor, Cliente).
- Catálogo de produtos com filtragem por árvore de categorias recursiva.
- Gestão de carrinho de compras persistente.
- Backoffice para Fornecedores (gestão de produtos próprios).
- Backoffice para Administradores (aprovação de produtos com definição de margem e gestão de encomendas).

7.2. Principais Dificuldades e Superação

Durante o ciclo de desenvolvimento, deparamos com desafios técnicos significativos, cuja resolução contribuiu para a qualidade final do projeto:

1. **Gestão de Concorrência de Stock:**
    - *Desafio:* Garantir que o stock não era descontado prematuramente (no *checkout*) nem permitia vendas acima da quantidade disponível.
    - *Solução:* Implementação de uma lógica transacional no método ExpedirEncomenda (API), que revalida o stock no momento exato da expedição e inativa automaticamente produtos que atinjam a quantidade zero.

2. **Relacionamentos Recursivos (Categorias):**
    - *Desafio:* Implementar um filtro que, ao selecionar uma categoria "Pai", mostrasse também os produtos das categorias "Filhas".
    - *Solução:* Desenvolvimento de um algoritmo recursivo no Backend que percorre a árvore de categorias para obter todos os IDs descendentes antes de executar a consulta SQL.

3. **Mapeamento Objeto-Relacional (EF Core):**
    - *Desafio:* Configuração correta das Chaves Estrangeiras (Foreign Keys) para a entidade ApplicationUser, dado que esta atua simultaneamente como Cliente (nas Encomendas) e Fornecedor (nos Produtos).
    - *Solução:* Configuração explícita no OnModelCreating do contexto de dados para definir os relacionamentos corretos e evitar ciclos de eliminação em cascata.

7.3. Análise Autocrítica

Numa perspetiva de melhoria contínua, identifica-se que a **Interface do Utilizador (UI)**, embora funcional e responsiva devido ao Bootstrap, beneficiaria de um maior investimento em *design* e usabilidade (UX), nomeadamente através de *feedback* visual mais rico (ex.: animações ao adicionar ao carrinho ou *toasts* de notificação).

Adicionalmente, a arquitetura atual, embora segura, delega muita responsabilidade ao Administrador para a manutenção da coerência dos dados. Num cenário de produção real, seria vantajoso implementar um sistema de **Logs de Auditoria** para registar quem alterou o stock ou aprovou determinado produto.

Em suma, o projeto "MyMedia" cumpre com distinção os requisitos de um sistema *Full-Stack*, demonstrando a capacidade de integração entre um Frontend reativo em Blazor e uma API RESTful complexa.